

Trabajo Práctico N° 2

SmithersTron (patente pendiente)



Fecha presentación	22/05/2025
Fecha entrega	12/06/2025

1 Introducción

El Sr. Burns recibió un informe hecho por Smithers que da visibilidad a la cantidad de accidentes que están ocurriendo últimamente en la planta nuclear. Con el objetivo de mejorar esta situación, el Sr. Burns nos pidió armar un sistema para registrar los incidentes ocurridos y asignárselos a sus empleados.

2 Objetivo

El presente trabajo práctico tiene como objetivo que el alumno:

- Se familiarice con el manejo de archivos en C.
- Realice apropiadamente las operaciones con archivos.
- Ponga en práctica el uso de argumentos por línea de comando.
- Se familiarice con y utilice correctamente memoria dinámica.
- Logre utilizar correctamente un TDA externo.

Por supuesto, se requiere que el trabajo cumpla con las **buenas prácticas de programación** profesadas por la cátedra. Se considerarán críticos la modularización, reutilización y claridad del código.

3 Enunciado

El objetivo del trabajo es crear un sistema que sirva para registrar, asignar y mostrar los incidentes ocurridos en la planta nuclear. El programa tendrá que recibir argumentos, utilizar un TDA ya creado por la cátedra y poder ejecutar una serie de comandos que ayudarán a administrar un reporte de incidentes en la planta.

3.1 Argumentos

El programa puede ser llamado de dos formas, con un argumento:

```
1 ./smitherstron <archivo_entrada>
```

O con dos argumentos:

```
1 ./smitherstron <archivo_entrada> <archivo_salida>
```

3.1.1 Formatos de los archivos

Los archivos serán de tipo CSV, guardarán una lista de incidentes **ordenada por prioridad**. Tendrán el siguiente formato:

```
1 <descripcion>;<hora>;<dia>;<prioridad>
```

- **descripcion**: Tiene como máximo 500 caracteres, se puede asumir que no contiene el caracter ‘;’.
- **hora**: Tiene el formato ‘HH:MM’ en formato de 24 horas. Solo se deben admitir horas válidas.
- **día**: Contiene la inicial del día en mayúscula (usando ‘X’ para el miércoles). Solo se deben admitir iniciales válidas.
- **prioridad**: Un número del 1 al 100 (donde 1 indica la mayor prioridad). Determinará cual es el orden en el que se irán asignando los incidentes. Si dos incidentes tienen el mismo valor de prioridad, el que ocurrió en un horario más temprano (sin importar el día) será el más prioritario.

3.1.2 Archivo de entrada

El programa recibirá por línea de comandos el nombre del archivo de entrada que contiene el reporte de todos los incidentes que ocurrieron. Cualquier problema con el formato (ya sea que no esté ordenado, o que los datos no sean válidos) debe detectarse y ser notificado como error.

3.1.3 Archivo de salida

Además el programa podrá, opcionalmente, recibir un segundo nombre de archivo que se usará para la salida, en caso de no especificarse significa que la salida sobrescribirá el archivo de entrada.

3.2 TDA Reporte

Para la ejecución del programa se contará con un TDA reporte que **ya estará implementado por la cátedra** y deberá ser utilizado (el código no va a estar disponible).

Las primitivas del TDA son las siguientes:

```

1 #ifndef __REPORTE_H__
2 #define __REPORTE_H__
3
4 #include <stdbool.h>
5
6 typedef struct incidente {
7     char *descripcion; // Un puntero a un string creado en el heap
8     int minutos_desde_medianoche; // Minutos pasadas la medianoche en el momento que ocurrió
9     // el incidente. Por ejemplo, si el incidente ocurrió las 6:23, significa que pasaron 383
10    minutos después de las 00:00. Este es el valor que se guardaría en ese caso.
11    char dia;
12    int prioridad;
13 } incidente_t;
14
15 struct reporte;
16 typedef struct reporte reporte_t;
17
18 // Pre: -
19 // Post: Crea un TDA reporte en el heap y devuelve un puntero al mismo.
20 // Devuelve NULL si no lo pudo crear.
21 reporte_t *reporte_crear();
22
23 // Pre: - El reporte fue previamente creado con `reporte_crear`.
24 //       - El incidente ya está completamente inicializado, incluido
25 //       el string dinámico.
26 // Post: Agrega un incidente **al final** del reporte.
27 void agregar_incidente(reporte_t *reporte, incidente_t incidente);
28
29 // Pre: - El reporte fue previamente creado con `reporte_crear`.
30 // Post: Elimina el incidente **al principio** del reporte.
31 //       Devuelve true si había incidentes para eliminar o false
32 //       si el reporte estaba vacío. Además devuelve el incidente
33 //       eliminado mediante la referencia `incidente_eliminado`.
34 bool eliminar_incidente(reporte_t *reporte, incidente_t *incidente_eliminado);
35
36 // Pre: El reporte recibido fue previamente creado con `reporte_crear`.
37 // Post: Libera la memoria que se reservó en el heap para el reporte.
38 void reporte_destruir(reporte_t *reporte);
39
40
41
42 #endif /* __REPORTE_H__ */

```

El TDA Reporte tiene funciones para agregar y quitar elementos pero de una forma particular, los elementos se sacan del reporte en el orden en el que ingresaron. Eso significa que si agrego tres elementos *A*, *B* y *C*. Se extraerán en ese orden, primero *A*, después *B* y finalmente *C*.

El TDA debe ser utilizado para guardar todos los incidentes que se procesarán a lo largo del programa. Los incidentes **no pueden ser guardados en vectores**, siempre tienen que guardarse en un `reporte_t` (excepto en el comando `agregar_multiples_incidentes` explicado más adelante).

3.3 Comandos

Al iniciarse el programa, se deberá cargar el reporte con los incidentes que están en el archivo de entrada (siempre y cuando sea un archivo válido), luego el usuario deberá ingresar un comando para ejecutar. Los comandos ejecutados realizarán operaciones sobre este reporte inicializado.

Cada comando realizará una acción distinta y tendrá parámetros distintos, los cuales estarán separados por ";". Después de cada comando el programa deberá volver a solicitar que el usuario ingrese un nuevo comando. Esto deberá continuar hasta que el usuario solicite el comando 'salir'.

Los parámetros de los comandos deberán ser validados, en caso de haber parámetros inválidos el comando no se ejecutará y el programa esperará un nuevo comando.

A continuación especificamos los comandos posibles:

3.3.1 `asignar_incidente`

Se utiliza para obtener el siguiente incidente a asignar. Este debe ser el de mayor prioridad. Se imprimirá por pantalla todos los detalles del incidente y se lo eliminará del reporte.

3.3.2 `agregar_incidente <descripción>;<hora>;<día>;<prioridad>`

Agrega un nuevo incidente al reporte, manteniendo el orden de prioridad en el mismo. En caso de recibir parámetros inválidos, el comando falla y no se solicitan que se vuelvan a ingresar.

3.3.3 `imprimir_incidentes`

Imprimirá todos los incidentes en el reporte en orden de prioridad.

3.3.4 `agregar_multiples_incidentes <n>`

Se usa para agregar varios incidentes en un solo comando. El parámetro n indica cuantos incidentes se quieren agregar. Después se le solicitará al usuario que ingrese `<descripción>;<hora>;<día>;<prioridad>` por cada incidente que se quiera agregar. Si cualquiera de los incidentes es inválido **se le solicitará al usuario que lo vuelva a ingresar**.

Este es el único lugar del TP donde pueden guardar los incidentes en un vector, de forma temporal mientras el usuario sigue ingresando los incidentes a agregar.

Al finalizar, se agregarán todos los incidentes al reporte, manteniendo el orden de prioridad.

3.3.5 `dia_mas_accidentado`

Imprime por pantalla cual es el día que más accidentes ocurrieron. El mensaje puede ser completamente personalizado pero el resultado final (osea el día) tiene que estar en mayúsculas y entre '*'. Por ejemplo, un mensaje válido podría ser "El día más peligroso es el *LUNES*".

3.3.6 `salir`

Indica que el usuario no quiere realizar más comandos, se tendrá que guardar el reporte de incidentes acualizado en el archivo especificado por línea de comando. El orden de la prioridad se debe mantener al guardarse el reporte en el archivo.

El archivo de salida depende de los argumentos mencionados anteriormente. Si solo se recibió un argumento, el archivo de entrada debe sobrescribirse con los nuevos cambios. Si se recibieron dos argumentos, el archivo de salida es el segundo especificado.

4 Resultado esperado

Esperamos que el trabajo cumpla la funcionalidad explicada anteriormente. Se deberá:

- Implementar todos los comandos mencionados.
- Utilizar correctamente el TDA provisto por la cátedra.
- Utilizar correctamente memoria dinámica, es decir, no tener pérdidas de memoria al ejecutarse el programa.
- Respetar las buenas prácticas de programación que profesamos en la cátedra.

5 Compilación y Entrega

El trabajo práctico debe ser realizado en un archivo llamado `smitherstron.c`

El trabajo debe ser compilado sin errores al correr el siguiente comando:

```
1 gcc *.c reporte.o -o smitherstron -std=c99 -Wall -Wconversion -Werror -lm
```

Por último debe ser entregado en la plataforma de corrección de trabajos prácticos **AlgoTrón** (patente pendiente), en la cual deberá tener la etiqueta **iExito!** significando que ha pasado las pruebas a las que la cátedra someterá al trabajo.

IMPORTANTE! *Esto no implica necesariamente haber aprobado el trabajo ya que además será corregido por un colaborador que verificará que se cumplan las buenas prácticas de programación. Para que el trabajo sea corregido por un colaborador, el mismo debe pasar todas las pruebas mínimas..*

Para la entrega en **AlgoTrón** (patente pendiente), recuerde que deberá subir un archivo **zip** conteniendo únicamente los archivos antes mencionados, sin carpetas internas ni otros archivos. De lo contrario, la entrega no será validada por la plataforma.

6 Anexo

6.1 FAQ

En este [link](#) encontrarán el documento de FAQ del TP, donde se irán cargando dudas realizadas con sus respuestas. Todo lo que esté en ese documento es válido y oficial para la realización del TP.